

Transformación y escalamiento de las variables

Issue #14 — MA2003B, equipo 7

Tabla de contenidos

1. Diagnóstico: ¿hace falta transformar?	3
1.1 Las escalas no son comparables	4
1.2 La asimetría es fuerte y sistemática	5
1.3 Ninguna variable es normal	5
2. Transformación de forma	6
2.1 Qué pide cada variable (Box-Cox)	6
2.2 El argumento que de verdad importa: linealidad	8
2.3 Qué NO se transforma, y por qué	9
3. Discretización	11
3.1 RAINF → ocurrencia de lluvia	11
3.2 hora y mes: codificación cíclica, no binning	11
3.3 La variable objetivo ya está discretizada (#13)	13
4. Atributos derivados: la tabla diaria	13
4.1 La forma cambia al agregar: hay que rediagnosticar	15
5. Escalamiento	16
5.1 Qué método	16
5.2 Verificación	18
5.3 Dos advertencias que valen para #15 y #17	19
6. Salidas	20
7. Resumen de decisiones	20
Referencias	22

Issue #14. Decide y aplica las transformaciones sobre el dataset limpio que dejó la issue #13 (data/processed/sima_limpio_horario.csv, $574,343 \times 31$, y sima_limpio_diario.csv, $23,931 \times 9$).

Este documento está escrito en **R** (motor **knitr**), a diferencia de [01_verificacion_calidad.qmd](#), que usa Python. Las dos mitades del equipo leen exactamente los mismos archivos de data/processed/, así que el lenguaje del análisis es indistinto.

La regla que sigue todo el documento: **no se transforma por costumbre**. Cada transformación se justifica con un número medido sobre estos datos, y las que se descartan también se documentan, con su razón.

```

library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(knitr)

knitr::opts_chunk$set(message = FALSE, warning = FALSE, fig.align = "center")

# El documento se puede renderizar con el cwd en notebooks/ o en la raiz del
# repo, segun como se lance. Esto lo resuelve en los dos casos.
RAIZ <- if (basename(getwd()) == "notebooks") dirname(getwd()) else getwd()
RUTA_PROCESSED <- file.path(RAIZ, "data", "processed")

exigir <- function(archivo) {
  ruta <- file.path(RUTA_PROCESSED, archivo)
  if (!file.exists(ruta)) {
    stop(sprintf("No existe%s.\nCorre primero:  python src/limpieza.py", ruta),
         call. = FALSE)
  }
  ruta
}

horario <- read_csv(exigir("sima_limpio_horario.csv"),
                    show_col_types = FALSE, guess_max = 5000)
diario <- read_csv(exigir("sima_limpio_diario.csv"), show_col_types = FALSE)

# La fecha viene sin zona horaria (hora local de pared). readr la lee como UTC,
# que es justo lo que se quiere: NO convertir, solo declarar. Ver cargar_datos.R.

# Variables medidas, en el orden del diccionario: objetivo, precursores,
# meteorologia.
MEDIDAS <- c("O3", "CO", "NO2", "NOX", "PM10", "PM2.5",
             "TOUT", "RAINF", "PRS", "WSR")

cat(sprintf("horario :%s x%s\n", format(nrow(horario), big.mark = ","),
      ncol(horario)))

```

horario : 574,343 x 31

```

cat(sprintf("diario :%s x%s\n", format(nrow(diario), big.mark = ","),
      ncol(diario)))

```

diario : 23,931 x 9

1. Diagnóstico: ¿hace falta transformar?

Las tres preguntas que deciden todo lo demás:

1. ¿Las variables están en escalas comparables? Si no, cualquier técnica basada en distancias o en la matriz de covarianza queda dominada por la de números más grandes.
2. ¿Qué tan lejos están de la simetría? La asimetría fuerte rompe los supuestos de las técnicas paramétricas y hace que la media deje de representar al centro.
3. ¿Las relaciones entre variables son lineales? PCA, análisis factorial y regresión **solo ven la parte lineal**; si la relación real es multiplicativa, la escala cruda esconde señal.

```
sesgo <- function(x) {  
  x <- x[!is.na(x)]  
  sum((x - mean(x))^3) / (length(x) * sd(x)^3)  
}  
  
curtosis <- function(x) {  
  x <- x[!is.na(x)]  
  sum((x - mean(x))^4) / (length(x) * sd(x)^4) - 3  
}  
  
resumen <- function(datos, vars) {  
  do.call(rbind, lapply(vars, function(v) {  
    x <- datos[[v]]  
    data.frame(  
      variable = v,  
      media    = mean(x, na.rm = TRUE),  
      sd       = sd(x, na.rm = TRUE),  
      min      = min(x, na.rm = TRUE),  
      max      = max(x, na.rm = TRUE),  
      cv       = sd(x, na.rm = TRUE) / abs(mean(x, na.rm = TRUE)),  
      sesgo    = sesgo(x),  
      curtosis = curtosis(x)  
    )  
  })))  
}  
  
tabla <- resumen(horario, MEDIDAS)  
kable(tabla, digits = 3, row.names = FALSE,  
      caption = "Estadísticos descriptivos del dataset horario limpio")
```

Tabla 1: Estadísticos descriptivos del dataset horario limpio

variable	media	sd	min	max	cv	sesgo	curtosis
O3	26.924	18.916	0.50	265.00	0.703	1.293	2.597
CO	1.356	0.864	0.00	37.00	0.637	1.815	16.075
NO2	14.814	11.193	0.00	167.80	0.756	1.932	6.887

variable	media	sd	min	max	cv	sesgo	curtosis
NOX	26.355	27.492	0.50	500.00	1.043	3.631	21.738
PM10	60.190	42.025	1.00	998.00	0.698	3.655	30.598
PM2.5	20.526	16.033	0.00	782.00	0.781	5.813	138.885
TOUT	23.231	7.423	-9.62	47.58	0.320	-0.418	0.007
RAINF	0.021	1.130	0.00	360.00	54.938	238.455	70720.835
PRS	715.330	9.399	655.40	750.00	0.013	0.102	-0.097
WSR	8.678	6.512	0.10	179.70	0.750	6.944	115.947

1.1 Las escalas no son comparables

PRS vive alrededor de 715 mmHg y CO alrededor de 1.36 ppm: **tres órdenes de magnitud de diferencia**. Pero el problema para un PCA no es la media, es la varianza. Este es el reparto de la varianza total cruda:

```
pesos <- sapply(horario[MEDIDAS], var, na.rm = TRUE)
pesos <- sort(100 * pesos / sum(pesos), decreasing = TRUE)

kable(data.frame(variable = names(pesos), pct_varianza_total = round(pesos, 2)),
      row.names = FALSE,
      caption = "Porcentaje de la varianza total que aporta cada variable en
      ↪ escala cruda")
```

Tabla 2: Porcentaje de la varianza total que aporta cada variable en escala cruda

variable	pct_varianza_total
PM10	51.19
NOX	21.91
O3	10.37
PM2.5	7.45
NO2	3.63
PRS	2.56
TOUT	1.60
WSR	1.23
RAINF	0.04
CO	0.02

PM10 y NOX concentran el 71 % de la varianza total. Un PCA sobre la matriz de covarianza de estos datos no descubriría estructura: devolvería una primera componente que es esencialmente PM10, y una segunda que es NOX. CO aporta 0.02 % — no porque sea irrelevante para el ozono, sino porque se mide en ppm y sus números son pequeños.

Estandarizar todas las variables equivale exactamente a correr el PCA sobre la **matriz de correlación** en vez de la de covarianza. Es la misma decisión dicha de dos maneras, y aquí es obligatoria: las variables tienen unidades distintas (ppb, ppm, $\mu\text{g}/\text{m}^3$, $^{\circ}\text{C}$, mmHg, km/h), así que la covarianza entre ellas no tiene interpretación física.

1.2 La asimetría es fuerte y sistemática

```
ggplot(tabla, aes(x = reorder(variable, sesgo), y = sesgo)) +  
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = -0.5, ymax = 0.5,  
    fill = "grey85") +  
  geom_col(fill = "steelblue") +  
  coord_flip() +  
  labs(x = NULL, y = "coeficiente de asimetría") +  
  theme_minimal()
```

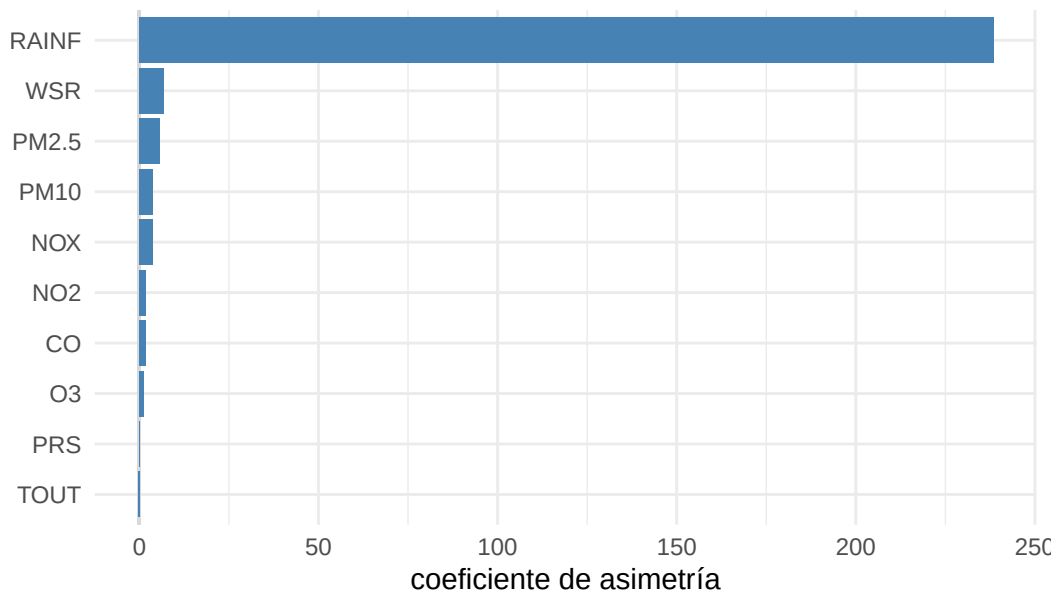


Figura 1: Asimetría por variable. La banda gris marca la zona $|\text{sesgo}| < 0.5$, donde la distribución se considera aproximadamente simétrica.

Salvo TOUT y PRS, todas las variables medidas están sesgadas a la derecha, y no por accidente: **una concentración es una cantidad no negativa con cola larga**. Hay muchas horas de aire limpio y pocas horas de episodio, y esas pocas horas se van muy lejos. PM2.5 llega a un sesgo de 5.8 y WSR a 6.9.

RAINF es un caso aparte, con sesgo de 238: no es una variable continua sesgada, es **una variable casi siempre cero**. Se trata en la §3.

1.3 Ninguna variable es normal

```
set.seed(2026) # el documento debe dar el mismo resultado en cada render  
  
# Shapiro-Wilk admite a lo mas 5,000 observaciones; con 574,343 filas cualquier  
# prueba de normalidad rechaza por tamaño de muestra, así que el p-valor aquí
```

```
# es descriptivo y el numero que importa es el estadistico W (1 = normal).
normalidad <- do.call(rbind, lapply(MEDIDAS, function(v) {
  x <- na.omit(horario[[v]])
  x <- sample(x, min(5000, length(x)))
  prueba <- shapiro.test(x)
  data.frame(variable = v, W = round(prueba$statistic, 4),
    p_valor = format.pval(prueba$p.value, digits = 3))
}))

kable(normalidad, row.names = FALSE,
  caption = "Shapiro-Wilk sobre una muestra de 5,000 observaciones por
  ↵ variable")
```

Tabla 3: Shapiro-Wilk sobre una muestra de 5,000 observaciones por variable

variable	W	p_valor
O3	0.9253	<2e-16
CO	0.9082	<2e-16
NO2	0.8618	<2e-16
NOX	0.6601	<2e-16
PM10	0.7331	<2e-16
PM2.5	0.8338	<2e-16
TOUT	0.9881	<2e-16
RAINF	0.0172	<2e-16
PRS	0.9950	3.53e-12
WSR	0.6047	<2e-16

Ninguna se acerca a la normalidad. TOUT y PRS son las que más se aproximan ($W \approx 0.98$), consistente con que son las dos únicas casi simétricas.

2. Transformación de forma

2.1 Qué pide cada variable (Box-Cox)

Box-Cox busca el exponente λ que más acerca la variable a la normalidad, maximizando la log-verosimilitud del perfil:

$$x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln x & \lambda = 0 \end{cases}$$

```

# Log-verosimilitud del perfil de Box-Cox para el modelo con solo intercepto.
# Se implementa a mano (8 lineas) en vez de usar MASS::boxcox para no agregar
# una dependencia al renv por una sola llamada.
log_verosimilitud_bc <- function(x, lambda) {
  n <- length(x)
  y <- if (abs(lambda) < 1e-8) log(x) else (x^lambda - 1) / lambda
  -n / 2 * log(sum((y - mean(y))^2) / n) + (lambda - 1) * sum(log(x))
}

lambda_optimo <- function(x, rejilla = seq(-2, 2, by = 0.05)) {
  x <- na.omit(x)
  x <- x[x > 0] # Box-Cox exige estrictamente positivo
  set.seed(2026)
  x <- sample(x, min(20000, length(x)))
  rejilla[which.max(sapply(rejilla, function(l) log_verosimilitud_bc(x, l)))]
}

SESGADAS <- c("O3", "CO", "NO2", "NOX", "PM10", "PM2.5", "WSR")

comparacion <- do.call(rbind, lapply(SESGADAS, function(v) {
  x <- horario[[v]]
  lam <- lambda_optimo(x)
  xp <- na.omit(x); xp <- xp[xp > 0]
  bc <- if (abs(lam) < 1e-8) log(xp) else (xp^lam - 1) / lam
  data.frame(variable = v,
             sesgo_crudo = sesgo(x),
             lambda_bc = lam,
             sesgo_bc = sesgo(bc),
             sesgo_log1p = sesgo(log1p(x)),
             sesgo_sqrt = sesgo(sqrt(x)))
}))

kable(comparacion, digits = 3, row.names = FALSE,
      caption = "Asimetría bajo cada transformación candidata")

```

Tabla 4: Asimetría bajo cada transformación candidata

variable	sesgo_crudo	lambda_bc	sesgo_bc	sesgo_log1p	sesgo_sqrt
O3	1.293	0.30	-0.068	-0.512	0.312
CO	1.815	0.30	-0.048	0.270	0.363
NO2	1.932	0.20	-0.010	-0.176	0.700
NOX	3.631	-0.10	-0.026	0.446	1.660
PM10	3.655	0.10	-0.065	-0.270	1.171
PM2.5	5.813	0.15	0.056	-0.121	1.031
WSR	6.944	0.30	0.120	-0.329	0.979

Los tres resultados que deciden:

1. **Todos los** caen entre **−0.10 y 0.35**, es decir, alrededor de 0. Los datos están pidiendo un logaritmo.
2. **Box-Cox y log1p dan resultados casi equivalentes**: ambos dejan el sesgo dentro de la banda $|\text{sesgo}| < 0.5$. La raíz cuadrada se queda corta en las más sesgadas (NOX pasa de 3.63 a 1.66, sigue fuera de banda).
3. **log1p gana por operatividad**. Box-Cox exige $x > 0$ y estas variables tienen ceros legítimos (NO2 y PM2.5 reportan 0 en horas limpias); descartarlos sería inventar faltantes. $\text{log1p}(x) = \ln(1+x)$ los admite, usa un solo para todas — así las variables siguen siendo comparables entre sí — y se revierte con `expm1`.

Decisión: log1p sobre las siete variables sesgadas.

2.2 El argumento que de verdad importa: linealidad

La normalidad es un beneficio secundario. La razón de fondo para el logaritmo es que las relaciones entre contaminantes son **multiplicativas**, y las técnicas del curso solo capturan la parte lineal:

```
lineal <- do.call(rbind, lapply(c("CO", "NO2", "NOX", "PM10", "PM2.5", "WSR"),
                              function(v) {
    data.frame(
      variable = v,
      pearson_crudo = cor(horario$O3, horario[[v]], use = "complete.obs"),
      pearson_log   = cor(log1p(horario$O3), log1p(horario[[v]]),
                          use = "complete.obs"),
      spearman      = cor(horario$O3, horario[[v]], use = "complete.obs",
                          method = "spearman")
    )
  }))

kable(lineal, digits = 3, row.names = FALSE,
      caption = "Correlación con O3: escala cruda vs. log-log, contra Spearman
      ↪ como referencia libre de escala")
```

Tabla 5: Correlación con O3: escala cruda vs. log-log, contra Spearman como referencia libre de escala

variable	pearson_crudo	pearson_log	spearman
CO	-0.075	-0.122	-0.107
NO2	-0.272	-0.359	-0.363
NOX	-0.329	-0.470	-0.434
PM10	0.019	-0.040	-0.002
PM2.5	0.000	-0.078	-0.045
WSR	0.274	0.448	0.482

Cómo se lee: **Spearman mide la asociación monótona sin importar la escala**, así que funciona como el techo de lo que se puede recuperar. En escala cruda, **pearson** se queda muy por debajo de Spearman; en escala log-log casi lo alcanza.

- NOX: $-0.329 \rightarrow -0.470$ (Spearman -0.434).
- WSR: $0.274 \rightarrow 0.448$ (Spearman 0.482).

Esa diferencia no es cosmética: es señal que el PCA y la regresión no habrían visto. El signo negativo de NOX frente al O3, por cierto, es el comportamiento esperado a nivel horario — el NO titula el ozono en las horas de tráfico.

2.3 Qué NO se transforma, y por qué

Variable	Decisión	Razón
TOUT	Sin transformar	Sesgo -0.42 , ya dentro de banda. Transformarla solo dificultaría leer los coeficientes, que están en °C.
PRS	Sin transformar	Sesgo 0.10 : es la variable más simétrica del conjunto.
viento_u, viento_v	Sin transformar	Son componentes vectoriales con signo : el negativo significa viento del este/sur. Cualquier transformación no lineal rompería la interpretación vectorial y la simetría alrededor de cero, que es justo lo que las hace utilizables.
WDR	No entra a los análisis	Es circular (0° y 359° son vecinos). Ya se substituyó por viento_u/viento_v en #13.
RAINF	Se discretiza	98.57 % de ceros: log1p deja el sesgo en 27.7, sigue inservible. Ver §3.1.

```
log_vars <- paste0("log_", SESGADAS)
horario[log_vars] <- lapply(horario[SESGADAS], log1p)

kable(data.frame(
  variable      = SESGADAS,
  sesgo_antes   = sapply(horario[SESGADAS], sesgo),
  sesgo_despues = sapply(horario[log_vars], sesgo),
  curtosis_antes = sapply(horario[SESGADAS], curtosis),
  curtosis_despues = sapply(horario[log_vars], curtosis)
), digits = 3, row.names = FALSE,
caption = "Efecto de log1p sobre asimetría y curtosis")
```

Tabla 7: Efecto de log1p sobre asimetría y curtosis

variable	sesgo_antes	sesgo_despues	curtosis_antes	curtosis_despues
O3	1.293	-0.512	2.597	-0.159
CO	1.815	0.270	16.075	-0.170
NO2	1.932	-0.176	6.887	0.096
NOX	3.631	0.446	21.738	0.284
PM10	3.655	-0.270	30.598	1.257
PM2.5	5.813	-0.121	138.885	-0.004
WSR	6.944	-0.329	115.947	0.349

```
comparar <- bind_rows(
  data.frame(valor = horario$`PM2.5`, escala = "PM2.5 (µg/m³)"),
  data.frame(valor = horario$log_PM2.5, escala = "log1p(PM2.5)")
)
comparar$escala <- factor(comparar$escala,
  levels = c("PM2.5 (µg/m³)", "log1p(PM2.5)"))

ggplot(comparar, aes(valor)) +
  geom_histogram(bins = 60, fill = "steelblue") +
  facet_wrap(~escala, scales = "free") +
  labs(x = NULL, y = "frecuencia") +
  theme_minimal()
```

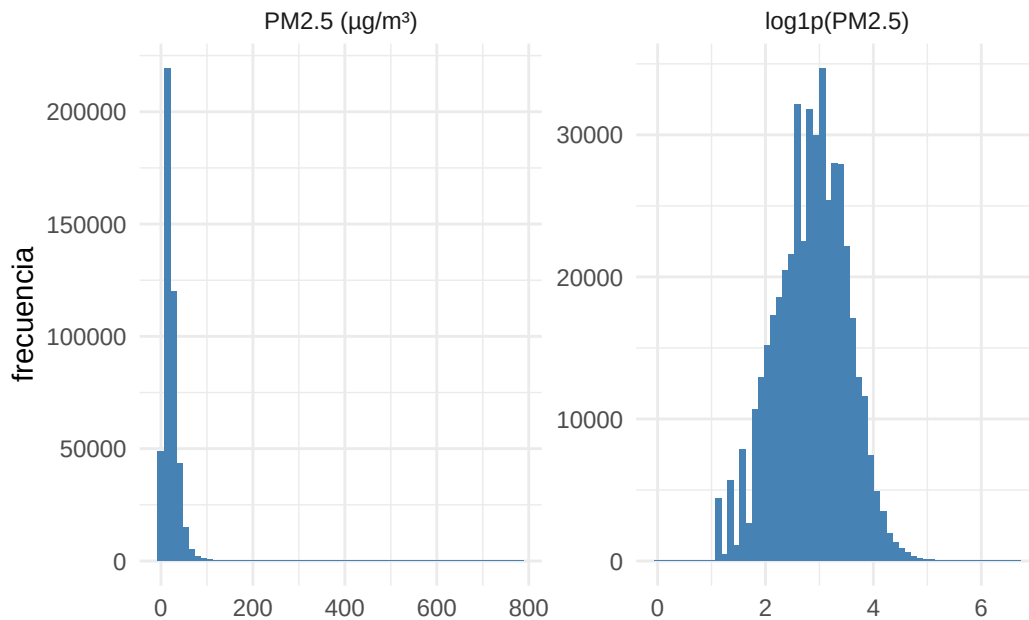


Figura 2: PM2.5 antes y después de log1p. La cola larga se convierte en una distribución trabajable.

3. Discretización

La issue pide evaluarla, no aplicarla en todos lados. Discretizar **destruye información** (dos valores distintos dentro del mismo bin se vuelven indistinguibles), así que solo se hace donde el bin significa algo.

3.1 RAINF → ocurrencia de lluvia

```
cat(sprintf("horas con RAINF == 0 : %.2f%%\n",
            100 * mean(horario$RAINF == 0, na.rm = TRUE)))
```

horas con RAINF == 0 : 98.57 %

```
cat(sprintf("sesgo crudo : %.1f\n", sesgo(horario$RAINF)))
```

sesgo crudo : 238.5

```
cat(sprintf("sesgo con log1p : %.1f\n", sesgo(log1p(horario$RAINF))))
```

sesgo con log1p : 27.7

```
horario$llovio <- as.integer(horario$RAINF > 0)
```

La lluvia horaria no es una magnitud continua sobre la que valga la pena hacer inferencia: es un **proceso de ocurrencia**. El mecanismo que interesa para el ozono tampoco es cuántos milímetros cayeron, sino que hubo nubes y depósito húmedo. Se conserva RAINF continua (por si algún análisis la quiere) y se agrega la binaria llovio.

3.2 hora y mes: codificación cíclica, no binning

hora ya es un entero 0–23, pero **como entero miente**: pone las 23:00 a 23 unidades de las 00:00 cuando son consecutivas. La respuesta correcta no es partirla en bloques sino proyectarla al círculo:

$$\text{hora}_{\sin} = \sin\left(\frac{2\pi h}{24}\right), \quad \text{hora}_{\cos} = \cos\left(\frac{2\pi h}{24}\right)$$

```

horario <- horario %>%
  mutate(
    hora_sin = sin(2 * pi * hora / 24),
    hora_cos = cos(2 * pi * hora / 24),
    mes_sin = sin(2 * pi * mes / 12),
    mes_cos = cos(2 * pi * mes / 12)
  )

# Verificacion: la distancia entre 23:00 y 00:00 debe ser igual a la que hay
# entre 00:00 y 01:00.
d <- function(a, b) {
  sqrt((sin(2*pi*a/24) - sin(2*pi*b/24))^2 + (cos(2*pi*a/24) - cos(2*pi*b/24))^2)
}
cat(sprintf("entero: |23 - 0| = %d      |0 - 1| = %d\n", 23L, 1L))

```

```
entero: |23 - 0| = 23      |0 - 1| = 1
```

```
cat(sprintf("ciclica: d(23, 0) = %.4f  d(0, 1) = %.4f\n", d(23, 0), d(0, 1)))
```

```
ciclica: d(23, 0) = 0.2611  d(0, 1) = 0.2611
```

Con la codificación cíclica las dos distancias coinciden, que es el comportamiento correcto. Se aplica el mismo tratamiento a `mes` (diciembre y enero son vecinos).

Como referencia interpretable se agrega además `periodo` — un binning explícito de la hora en las cuatro fases del ciclo fotoquímico diario. **No entra a los modelos**, sirve para tablas y gráficas:

```

horario$periodo <- cut(horario$hora,
                      breaks = c(-1, 5, 11, 17, 23),
                      labels = c("madrugada", "mañana", "tarde", "noche"))

horario %>%
  group_by(periodo) %>%
  summarise(O3_medio = mean(O3, na.rm = TRUE),
            NOX_medio = mean(NOX, na.rm = TRUE), .groups = "drop") %>%
  kable(digits = 2, caption = "El ciclo diurno: el O3 se forma en la tarde, cuando
  ↪ el NOX ya bajó")

```

Tabla 8: El ciclo diurno: el O3 se forma en la tarde, cuando el NOX ya bajó

periodo	O3_medio	NOX_medio
madrugada	17.58	23.85
mañana	17.86	35.51
tarde	45.45	20.10

periodo	O3_medio	NOX_medio
noche	26.58	25.99

3.3 La variable objetivo ya está discretizada (#13)

El objetivo del equipo es **clasificar excedencias**, así que el binning del MDA8 no es una decisión libre: lo fija la NOM-020-SSA1-2021. #13 ya dejó las tres columnas indicadoras; aquí solo se revisa el balance de clases, que es lo que condiciona la técnica de #15.

```
balance <- diario %>%
  summarise(across(c(excede_anio_1, excede_anio_3, excede_anio_5),
    ~ 100 * mean(.x, na.rm = TRUE))) %>%
  pivot_longer(everything(), names_to = "umbral", values_to = "pct_excedencias")
  ↪ %>%
  mutate(ppb = c(65, 60, 51), .after = umbral)

kable(balance, digits = 2,
  caption = "Balance de clases por umbral de la NOM-020-SSA1-2021")
```

Tabla 9: Balance de clases por umbral de la NOM-020-SSA1-2021

umbral	ppb	pct_excedencias
excede_anio_1	65	10.65
excede_anio_3	60	16.04
excede_anio_5	51	30.70

Nota para #15: con el umbral vigente (65 ppb) solo 10.65 % de los días-estación son excedencia. Un clasificador que responda siempre “no excede” acierta 89 % y no sirve de nada, así que la exactitud no puede ser la métrica de evaluación. El umbral de 51 ppb (30.7 %) está mucho mejor balanceado y conviene usarlo como escenario principal de modelado.

4. Atributos derivados: la tabla diaria

Este es el punto más importante del documento. #13 dejó el limpio a resolución **horaria**, pero el objetivo del equipo (MDA8, excedencias) es **diario**. Las dos tablas no se pueden cruzar tal cual: hay que agregar los predictores a día-estación, y *cómo* se agrega cada uno depende de su mecanismo físico.

```
maximo_seguro <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) NA_real_ else max(x)
}
```

```

predictores_diarios <- horario %>%
  mutate(dia = as.Date(fecha)) %>%
  group_by(estacion, dia) %>%
  summarise(
    # Meteorologia: la temperatura MAXIMA es la que gobierna la fotoquimica,
    # no la media del dia.
    TOUT_max = maximo_seguro(TOUT),
    PRS_med  = mean(PRS, na.rm = TRUE),
    WSR_med  = mean(WSR, na.rm = TRUE),
    u_med    = mean(viento_u, na.rm = TRUE),
    v_med    = mean(viento_v, na.rm = TRUE),
    # Lluvia: acumulado del dia y ocurrencia.
    RAINF_sum = sum(RAINF, na.rm = TRUE),
    llovio    = as.integer(any(RAINF > 0, na.rm = TRUE)),
    # Precusores: la ventana 06-09 h es la emision matutina del trafico, que
    # es la que alimenta el ozono de la tarde. Promediar las 24 h diluiria
    # justo la senal que interesa.
    NOX_am    = mean(NOX[hora >= 6 & hora <= 9], na.rm = TRUE),
    NO2_am    = mean(NO2[hora >= 6 & hora <= 9], na.rm = TRUE),
    CO_am     = mean(CO[hora >= 6 & hora <= 9], na.rm = TRUE),
    PM10_med  = mean(PM10, na.rm = TRUE),
    PM25_med  = mean(`PM2.5`, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  # mean() de un grupo totalmente vacio devuelve NaN y sum() devuelve 0; se
  # normaliza a NA para que los faltantes se cuenten como faltantes.
  mutate(across(where(is.numeric), ~ ifelse(is.finite(.x), .x, NA_real_)))

trabajo <- diario %>%
  mutate(dia = as.Date(fecha)) %>%
  inner_join(predictores_diarios, by = c("estacion", "dia")) %>%
  mutate(
    mes      = as.integer(format(dia, "%m")),
    mes_sin  = sin(2 * pi * mes / 12),
    mes_cos  = cos(2 * pi * mes / 12),
    dia_semana = as.integer(format(dia, "%u")) - 1L,
    fin_de_semana = as.integer(dia_semana >= 5)
  )

cat(sprintf("tabla de trabajo diaria: %s x %s\n",
            format(nrow(trabajo), big.mark = ","), ncol(trabajo)))

```

tabla de trabajo diaria: 23,931 x 27

Por qué cada agregación:

Atributo	Agregación	Mecanismo
TOUT_max	Máximo del día	La formación fotoquímica escala con la temperatura del pico, no con el promedio diario.
NOX_am, NO2_am, CO_am	Media 06–09 h	Los precursores emitidos en la hora punta matutina son los que reaccionan y producen el MDA8 de la tarde. Promediar 24 h mezclaría esa emisión con la titulación nocturna y cancelaría la señal.
WSR_med, u_med, v_med	Media del día	La ventilación es un efecto acumulado sobre la masa de aire.
RAINF_sum, llovio	Suma / indicadora	Acumulado del día más ocurrencia.
PM10_med, PM25_med	Media del día	Actúan como pantalla contra la radiación durante toda la jornada.

4.1 La forma cambia al agregar: hay que rediagnosticar

```
NUM_DIARIAS <- c("MDA8", "TOUT_max", "PRS_med", "WSR_med", "u_med", "v_med",
                 "RAINF_sum", "NOX_am", "NO2_am", "CO_am", "PM10_med", "PM25_med")

kable(data.frame(
  variable = NUM_DIARIAS,
  sesgo    = sapply(trabajo[NUM_DIARIAS], sesgo),
  curtosis = sapply(trabajo[NUM_DIARIAS], curtosis)
), digits = 3, row.names = FALSE,
caption = "Asimetría de las variables ya agregadas a día-estación")
```

Tabla 11: Asimetría de las variables ya agregadas a día-estación

variable	sesgo	curtosis
MDA8	0.643	0.968
TOUT_max	-0.839	0.564
PRS_med	0.109	-0.127
WSR_med	8.084	137.666
u_med	-1.799	26.625
v_med	-1.956	33.325
RAINF_sum	95.742	11846.173
NOX_am	2.471	9.600
NO2_am	1.705	7.174

variable	sesgo	curtosis
CO_am	1.298	4.096
PM10_med	2.368	26.901
PM25_med	2.144	10.900

Dos hallazgos:

- **MDA8 tiene sesgo 0.64: casi simétrica, y no se transforma.** Es el promedio móvil de 8 h del máximo diario, y promediar 8 valores ya hace la mitad del trabajo (teorema del límite central). Transformar la variable objetivo complicaría la interpretación — los coeficientes dejarían de estar en ppb, que es la unidad en la que está escrita la norma — sin ganar nada.
- **Los precursores agregados siguen sesgados** (NOX_am 2.47, PM10_med 2.37, WSR_med 8.08), así que la decisión de la §2 se mantiene en el nivel diario.

```
LOG_DIARIAS <- c("WSR_med", "NOX_am", "NO2_am", "CO_am", "PM10_med", "PM25_med",
  "RAINF_sum")
trabajo[paste0("log_", LOG_DIARIAS)] <- lapply(trabajo[LOG_DIARIAS], log1p)

kable(data.frame(
  variable      = LOG_DIARIAS,
  sesgo_antes   = sapply(trabajo[LOG_DIARIAS], sesgo),
  sesgo_despues = sapply(trabajo[paste0("log_", LOG_DIARIAS)], sesgo)
), digits = 3, row.names = FALSE,
caption = "log1p en el nivel diario")
```

Tabla 12: log1p en el nivel diario

variable	sesgo_antes	sesgo_despues
WSR_med	8.084	-0.147
NOX_am	2.471	0.100
NO2_am	1.705	-0.624
CO_am	1.298	0.191
PM10_med	2.368	-0.331
PM25_med	2.144	0.027
RAINF_sum	95.742	9.687

RAINF_sum se transforma aquí (y no en el horario) porque al acumular el día el porcentaje de ceros baja de 98.6 % a 92 %, pero **sigue sin ser utilizable como continua**: para modelar, la que sirve es llovio.

5. Escalamiento

5.1 Qué método

Método	Fórmula	Cuándo	Aquí
z-score	$(x - \bar{x})/s$	Técnicas basadas en covarianza/correlación: PCA, factorial, discriminante	Elegido
Min-max	$(x - \text{mín})/(\text{máx} - \text{mín})$	Cuando se necesita un rango acotado [0,1]	Descartado
Robusto	$(x - \text{mediana})/\text{IQR}$	Con outliers extremos que no se pueden tocar	Descartado

Se elige z-score. Las técnicas que vienen en #15 y #17 (PCA, análisis factorial, conglomerados, discriminante) operan sobre la matriz de correlación, y estandarizar *es* trabajar con ella.

Min-max se descarta porque lo fijan el mínimo y el máximo, que son precisamente los valores más frágiles de estos datos: #13 documentó que hay outliers conservados a propósito (WSR llega a 179.7 km/h, PM10 a 998 $\mu\text{g}/\text{m}^3$). Un solo valor extremo comprimiría todo el resto de la variable en una franja diminuta del [0,1].

Robusto se descarta por una razón de fondo del proyecto: los outliers de este dataset **no son ruido, son el fenómeno**. El objetivo es clasificar episodios de concentración alta; un escalador diseñado para restarle peso a la cola superior trabajaría en contra del objetivo. Además el `log1p` de la §2 ya redujo la influencia de la cola, que es de donde venía el problema.

Orden de las operaciones: primero `log1p`, después z-score. Al revés no funciona: el z-score produce valores negativos y el logaritmo ya no se puede aplicar.

```
estandarizar <- function(datos, vars) {
  centros <- sapply(datos[vars], mean, na.rm = TRUE)
  escalas <- sapply(datos[vars], sd, na.rm = TRUE)
  z <- as.data.frame(scale(datos[vars], center = centros, scale = escalas))
  names(z) <- paste0("z_", vars)
  list(datos = z,
       parametros = data.frame(variable = vars, centro = centros,
                               escala = escalas, row.names = NULL))
}

A_ESCALAR <- c("MDA8", "TOUT_max", "PRS_med", "u_med", "v_med",
               paste0("log_", c("WSR_med", "NOX_am", "NO2_am", "CO_am",
                                "PM10_med", "PM25_med"))))

esc <- estandarizar(trabajo, A_ESCALAR)
trabajo <- bind_cols(trabajo, esc$datos)

kable(esc$parametros, digits = 4,
      caption = "Parámetros de estandarización. Se guardan en data/processed/ para
↳ poder revertir la transformación e interpretar resultados en unidades
↳ originales.")
```

Tabla 14: Parámetros de estandarización. Se guardan en data/processed/ para poder revertir la transformación e interpretar resultados en unidades originales.

variable	centro	escala
MDA8	44.6097	16.4169
TOUT_max	28.7928	7.0816
PRS_med	715.3230	9.3241
u_med	-4.2292	5.1771
v_med	0.2366	4.3904
log_WSR_med	2.1945	0.3806
log_NOX_am	3.3722	0.7405
log_NO2_am	2.7290	0.5934
log_CO_am	0.8284	0.3373
log_PM10_med	4.0063	0.4732
log_PM25_med	2.9479	0.4877

5.2 Verificación

```
verifica <- data.frame(
  variable = A_ESCALAR,
  media = sapply(trabajo[paste0("z_", A_ESCALAR)], mean, na.rm = TRUE),
  sd     = sapply(trabajo[paste0("z_", A_ESCALAR)], sd, na.rm = TRUE)
)
kable(verifica, digits = 6, row.names = FALSE,
      caption = "Media 0 y desviación 1 en todas las variables escaladas")
```

Tabla 15: Media 0 y desviación 1 en todas las variables escaladas

variable	media	sd
MDA8	0	1
TOUT_max	0	1
PRS_med	0	1
u_med	0	1
v_med	0	1
log_WSR_med	0	1
log_NOX_am	0	1
log_NO2_am	0	1
log_CO_am	0	1
log_PM10_med	0	1
log_PM25_med	0	1

```
pesos_z <- sapply(trabajo[paste0("z_", A_ESCALAR)], var, na.rm = TRUE)
cat("Reparto de la varianza total tras estandarizar (%):\n")
```

Reparto de la varianza total tras estandarizar (%):

```
print(round(100 * pesos_z / sum(pesos_z), 2))
```

z_MDA8	z_TOUT_max	z_PRS_med	z_u_med	z_v_med
9.09	9.09	9.09	9.09	9.09
z_log_WSR_med	z_log_NOX_am	z_log_NO2_am	z_log_CO_am	z_log_PM10_med
9.09	9.09	9.09	9.09	9.09
z_log_PM25_med				
9.09				

Comparar con la tabla de la §1.1: se pasó de que PM10 y NOX acapararan el 71 % de la varianza, a un reparto uniforme donde cada variable pesa lo mismo. Eso es exactamente lo que se buscaba: que el PCA encuentre estructura y no el artefacto de las unidades.

5.3 Dos advertencias que valen para #15 y #17

(a) **No centrar dentro de cada estación.** Es una tentación razonable — quitaría el efecto de sitio — pero los números no la respaldan:

```
modelo <- aov(MDA8 ~ estacion, data = trabajo)
tabla_anova <- summary(modelo)[[1]]
sc_entre <- tabla_anova[["Sum Sq"]][1]
sc_dentro <- tabla_anova[["Sum Sq"]][2]

cat(sprintf("F = %.1f   (p < 2e-16)\n", tabla_anova[["F value"]][1]))
```

F = 89.8 (p < 2e-16)

```
cat(sprintf("varianza explicada por la estación: %.1f%%\n",
            100 * sc_entre / (sc_entre + sc_dentro)))
```

varianza explicada por la estación: 5.3 %

La estación sí importa ($F = 89.8$, altamente significativa), pero explica solo **5.3 % de la varianza del MDA8**. Centrar por estación borraría ese contraste espacial a cambio de casi nada — y ese contraste es justamente lo que el análisis de conglomerados debe descubrir. **La estación se usa para validar el agrupamiento obtenido, no como corrección previa.**

(b) **Los parámetros de escalamiento se calculan sobre el conjunto de entrenamiento, no sobre todo el dataset.** Aquí se estandariza con la media y la desviación de las 23,931 filas porque este documento describe los datos completos. En cuanto #15 parta en entrenamiento y prueba, la media y la desviación deben salir **solo del entrenamiento** y aplicarse a la prueba: si no, la prueba filtra información sobre su propia distribución al modelo. Por eso `parametros_escalamiento.csv` se guarda aparte y no queda enterrado en el dataset.

6. Salidas

```
escribir <- function(datos, archivo) {  
  ruta <- file.path(RUTA_PROCESSED, archivo)  
  write_csv(datos, ruta)  
  cat(sprintf("%-38s%s filas x%s columnas\n", archivo,  
              format(nrow(datos), big.mark = ","), ncol(datos)))  
}  
  
escribir(trabajo, "sima_transformado_diario.csv")
```

sima_transformado_diario.csv 23,931 filas x 45 columnas

```
escribir(horario, "sima_transformado_horario.csv")
```

sima_transformado_horario.csv 574,343 filas x 44 columnas

```
escribir(esc$parametros, "parametros_escalamiento.csv")
```

parametros_escalamiento.csv 11 filas x 3 columnas

- **sima_transformado_diario.csv** — la tabla de trabajo para #15 y #17: una fila por día-estación, con MDA8, las indicadores de excedencia, los predictores agregados, sus versiones `log_` y sus versiones `z_`.
- **sima_transformado_horario.csv** — el horario con las columnas `log_`, las cíclicas y `llovio`, por si algún análisis necesita la resolución fina.
- **parametros_escalamiento.csv** — centro y escala de cada variable, para revertir y para reportar resultados en unidades originales.

Ninguno se versiona (`data/processed/` está en `.gitignore`); se regeneran corriendo `python src/limpieza.py` y volviendo a renderizar este documento.

7. Resumen de decisiones

Sub-punto de la issue	Decisión	Justificación
Discretización	Aplicada de forma selectiva	RAINF $\rightarrow$ llovio (98.6 % ceros); hora $\rightarrow$ periodo solo para reportar. El binning del MDA8 ya lo fija la NOM y viene de #13. No se discretiza ningún otro continuo: destruiría información sin ganar interpretabilidad.
Escalamiento	z-score , aplicado	PM10 y NOX acaparaban el 71 % de la varianza cruda; sin estandarizar, el PCA devolvería las unidades y no la estructura. Min-max y robusto se descartan por los outliers conservados en #13.
Transformación de forma	log1p en 7 variables horarias y 7 diarias	Box-Cox pide $[-0.10, 0.35]$ en todas, o sea el logaritmo; log1p además admite los ceros. Sube $r(O3, NOX)$ de -0.33 a -0.47 y $r(O3, WSR)$ de 0.27 a 0.45 .
Atributos derivados	Tabla diaria de predictores + cíclicas sin/cos	El objetivo es diario y el limpio era horario: sin la agregación no hay dataset de modelado. La ventana 06–09 h de precursores y TOUT_max responden al mecanismo fotoquímico, no a conveniencia.
Transformar el objetivo MDA8	Omitida	Sesgo 0.64, ya simétrica; transformarla sacaría los coeficientes de ppb, la unidad de la norma.
Transformar TOUT, PRS	Omitida	Sesgos -0.42 y 0.10 : ya simétricas.
Transformar viento_u, viento_v	Omitida	Componentes con signo; una transformación no lineal rompería la interpretación vectorial.
Centrado por estación	Omitida	La estación explica 5.3 % de la varianza del MDA8 ; centrarla borraría el contraste espacial que #15 debe descubrir.

Sub-punto de la issue	Decisión	Justificación
Dummies de estacion/temporada	Omitida (sigue vigente lo de #13)	Depende de la técnica: PCA y conglomerados no las quieren, regresión y discriminante sí. Se generan en #15, donde ya se sabe cuál se usa.

Referencias

- Dataset limpio y sus reglas: [docs/log_limpieza.md](#) (#13)
- Variables seleccionadas: [docs/seleccion_datos.md](#) (#12)
- Diccionario y unidades: [docs/Diccionario_Sima.md](#)
- NOM-020-SSA1-2021: límite de 8 h, 0.065 ppm (65 ppb)